

Reviewer Pack — Minimal Root Separation of Polynomials over Number Fields vs...

Entry 3f765a1f3567 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-22 08:51:14 UTC

Minimal Root Separation of Polynomials over Number Fields vs AC0 Parity Threshold

Entry ID: 3f765a1f3567 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-22 08:51:14 UTC

1. Conjecture

Field A (mathematical branch): Number Theory: Number Fields **Field B** (complexity object): Complexity Theory: AC0 Parity Circuit Complexity

Statement:

{‘sentence1’: ‘For any polynomial $P(x)$ over a number field F with degree $d \geq 3$, the minimal distance between distinct roots of P is at least $c \cdot \log(d)$ ’, ‘sentence2’: ‘This lower bound holds for any constant $c > 0$ and implies that any AC0 circuit computing the parity of n inputs requires at least $1.5c \cdot n$ gates.’, ‘sentence3’: ‘The conjecture suggests a connection between algebraic properties of number fields and complexity-theoretic measures of computational difficulty.’}

Rationale (proposer’s reasoning):

{‘sentence1’: ‘Minimal root separation in number theory is closely related to the transcendence degree of an extension field, which has been used in studying the complexity of certain problems.’, ‘sentence2’: ‘The AC0 parity threshold problem is a fundamental question in circuit complexity that remains unsolved, and understanding its relation to algebraic properties could shed light on its difficulty.’, ‘sentence3’: ‘This conjecture proposes a new direction for exploring this connection, potentially leading to novel techniques for proving lower bounds.’}

Taxonomy category: AC0_PARITY (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 5e4356276d53e1ba

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

Determine if a polynomial $P(x)$ over a number field F with degree $d \geq 3$ has distinct roots with a minimal distance of at least $c \cdot \log(d)$, where $c > 0$, and correlate this with an AC0 parity circuit requiring at least $1.5c \cdot n$ gates for n inputs.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	1.00	HITS	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def generate_polynomial(d, F):
    coefficients = [random.choice(F) for _ in range(d + 1)]
    x = symbols('x')
    return sum(c * x**i for i, c in enumerate(coefficients))

def find_roots(P):
    # Use a simple numerical method to find roots
    def f(x):
        return P.subs(x, x)

    roots = []
    for _ in range(10): # Try up to 10 times
        guess = random.uniform(-10, 10)
        root = newton_method(f, guess, max_iter=100)
        if root not in roots:
            roots.append(root)
    return roots

def newton_method(f, x0, tol=1e-6, max_iter=100):
    for _ in range(max_iter):
        fx = f(x0)
        dfx = f.diff(x).subs(x, x0)
        if abs(dfx) < tol:
            break
        x0 -= fx / dfx
    return x0

def run_trial(seed: int) -> dict:
    random.seed(seed)
```

```

# Define the number field F and degree d
F = [Fraction(1), Fraction(-1)] # Example number field Q(i)
d = random.randint(3, 40)

# Generate a polynomial P(x) over the number field F with degree d
P = generate_polynomial(d, F)

# Find the roots of the polynomial
roots = find_roots(P)

# Calculate the minimal distance between distinct roots
distances = [abs(roots[i] - roots[j]) for i in range(len(roots)) for j in range(i + 1, len(roots))]
min_distance = min(distances) if distances else float('inf')

# Calculate the gate count of an AC0 parity circuit for n inputs
n = len(roots)
c = random.uniform(0.1, 1.0) # Random constant c > 0
mean_circuit_size = 1.5 * c * n

# Check if the conjecture holds
conjecture_holds = min_distance >= c * math.log(d)

return {
    "metric_name": "Minimal Root Distance",
    "metric_value": min_distance,
    "instances_tested": len(roots),
    "conjecture_holds": conjecture_holds,
    "counterexample": "" if conjecture_holds else f"Counterexample with d={d}, c={c}"
}

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or list(range(2, 30)) # Default to first 29 primes if

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_value = sum(r["metric_value"] for r in results) / len(results)
    std_value = math.sqrt(sum((r["metric_value"] - mean_value) ** 2 for r in results) / len(results))
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
    else:
        first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample='min_distance < c*log(d)' first_failing_seed={first_failing_seed}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

--STDERR--

Traceback (most recent call last):

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_f05a65f5.py", line 84, in <module>
    result = run_trial(seed)
            ^^^^^^^^^^^^^^^
```

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_f05a65f5.py", line 53, in run_trial
    P = generate_polynomial(d, F)
      ^^^^^^^^^^^^^^^^^^^^^^^
```

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_f05a65f5.py", line 20, in generate_poly
    x = symbols('x')
      ^^^^^^^
```

NameError: name 'symbols' is not defined

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test code crashed before producing data, which prevents us from determining if the conjecture is supported or falsified. | next: Debug the test code to ensure it runs successfully and produces results.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	15331
2	preregistration	ollama_remote	glm4:latest	0	0	9733
3	novelty	ollama_remote	glm4:latest	0	0	8516
4	novelty	ollama_remote	glm4:latest	0	0	8247
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14076
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11400
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9481
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10278
9	judge	ollama_remote	glm4:latest	0	0	11371

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 98434 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/3f765a1f3567.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/3f765a1f3567.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/3f765a1f3567.tar.gz` (if generated)